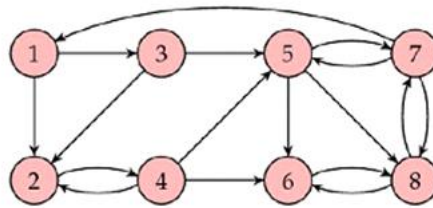


2. Dato il seguente grafo



deriva la matrice G di connettività e formula il problema del pagerank. Risolvi il problema utilizzando i codici Matlab a tua disposizione. Qual è il ranking delle pagine?

MATLAB R2020b - academic use

EDITOR - G:\Il mio Drive\Università\didattica\Didattica2024-2025\Informatica\cs inf\programr

esercizioPageRank1.m

```

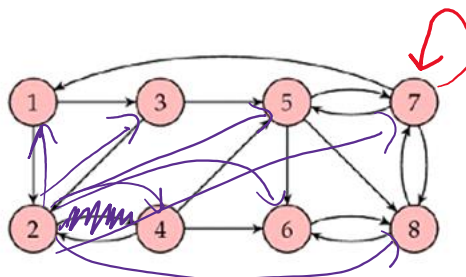
11 % ESERCIZIO 2
12 G=[0 0 0 0 0 0 1 0
13     1 0 1 1 0 0 0 0
14     1 0 0 0 0 0 0 0
15     0 1 0 0 0 0 0 0
16     0 0 1 1 0 0 1 0
17     0 0 0 1 1 0 0 1
18     0 0 0 0 1 0 0 1
19     0 0 0 0 1 1 1 0];
20 n=size(G,1); p=0.85; delta=(1-p)/n;
21 I=eye(n);
22 c=sum(G,1);
23 D=diag(1./c);
24 e=ones(n,1);
25 A=I-p*G*D;
26 b=delta*e;
27 x=A\b
  
```

Workspace:

Name	Value
x	0.0631 0.0925 0.0456 0.0974 0.1101 0.1841 0.1565 0.2508
sum(x)	1.0000

fx >>

2. Dato il seguente grafo



deriva la matrice G di connettività e formula il problema del pagerank. Risolvi il problema utilizzando i codici Matlab a tua disposizione. Qual è il ranking delle pagine?

The image shows the MATLAB R2020b interface. The Command Window on the left displays the following output:

```
>> esercizioPageRank1
Warning: Matrix is singular, close to zero. RCOND = NaN.
> In esercizioPageRank1 (line 27)

x =

NaN
NaN
NaN
NaN
NaN
NaN
NaN
NaN
NaN
NaN
```

The Editor window on the right shows the script 'esercizioPageRank1.m' with the following code:

```
11 % Esercizio 2
12 G=[0 0 0 0 0 0 1 0
13     1 0 1 1 0 0 0 0
14     1 0 0 0 0 0 0 0
15     0 0 0 0 0 0 0 0
16     0 0 1 1 0 0 1 0
17     0 0 0 1 1 0 0 1
18     0 0 0 0 1 0 1 1
19     0 0 0 0 1 1 1 0];
20 n=size(G,1); p=0.85; delta=(1-p)/n;
21 I=eye(n);
22 c=sum(G,1);
23 D=diag(1./c);
24 e=ones(n,1);
25 A=I-p*G*D;
26 b=delta*e;
27 x=A\b
```

Handwritten annotations in the script include a red '1' in line 18 and a purple 'c2=7' next to line 25.

$$(1) \quad x_i = p \left(g_{i1} \frac{x_1}{c_1} + g_{i2} \frac{x_2}{c_2} + \dots + g_{in} \frac{x_n}{c_n} \right) + \frac{1}{n} (1-p)$$

$$D = \begin{pmatrix} \frac{1}{c_1} & & \\ & \frac{1}{c_2} & \\ & & \ddots \end{pmatrix}$$

Handwritten notes include an arrow pointing from the circled c_2 in the equation above to the $\frac{1}{c_2}$ in the matrix, and the text "= 0 -> 7" next to it.

NOTE ALGORITMICHE: CASO $c_j=0$

ossia dalla pagina j non partono link

Si fa in modo che l'importanza della pagina j sia uniformemente distribuita a tutte le pagine:

- Si pone $g_{ij}=1$ per ogni i diverso da j
- perciò si ha $c_j = n-1$, e di conseguenza nell'equazione di Google risulta:

$$x_i = p \left(g_{i1} \frac{x_1}{c_1} + \dots + \frac{x_j}{n-1} + \dots + g_{in} \frac{x_n}{c_n} \right) + \frac{1}{n}(1-p), \forall i$$

da cui è evidente che ad ogni pagina i è trasferita la stessa quota di importanza della pagina j

7

The screenshot displays the MATLAB R2020b environment. The Editor window shows a script named `esercizioPageRank2.m` with the following code:

```
11 % ESERCIZIO
12 G=[0 0 0 0 0 0 1 0
13     1 0 1 1 0 0 0 0
14     1 0 0 0 0 0 0 0
15     0 0 0 0 0 0 0 0
16     0 0 1 1 0 0 1 0
17     0 0 0 1 1 0 0 1
18     0 0 0 0 1 0 1 1
19     0 0 0 0 1 1 1 0];
20 n=size(G,1); p=0.85; delta=(1-p)/n;
21 I=eye(n);
22
23 G=G-diag(diag(G)); %eliminare eventuali link aut
24 c=sum(G);
25 index=find(c==0); ni=length(index);
26 c(index)=(n-1)*ones(1,ni);
27 G(:,index)=ones(n,ni);
28 G=G-diag(diag(G)); %eliminare eventuali link au
29
30
31 D=diag(1./c);
32 e=ones(n,1);
33 A=I-p*G*D;
34 b=delta*e;
35 x=A\b
```

The Command Window shows the execution of the script, displaying the value of `x`:

```
>> esercizioPageRank2.m
>> esercizioPageRank2.m
x =
    0.0791
    0.0875
    0.0630
    0.0294
    0.1142
    0.1838
    0.1755
    0.2677
```

A separate window shows the value of the variable `G`:

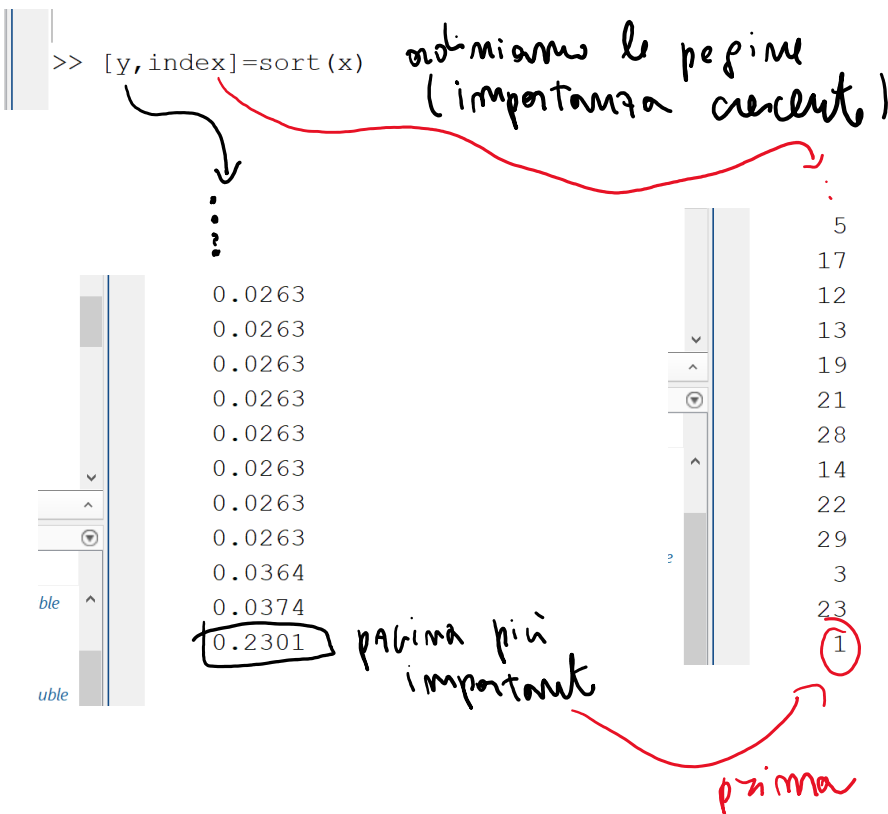
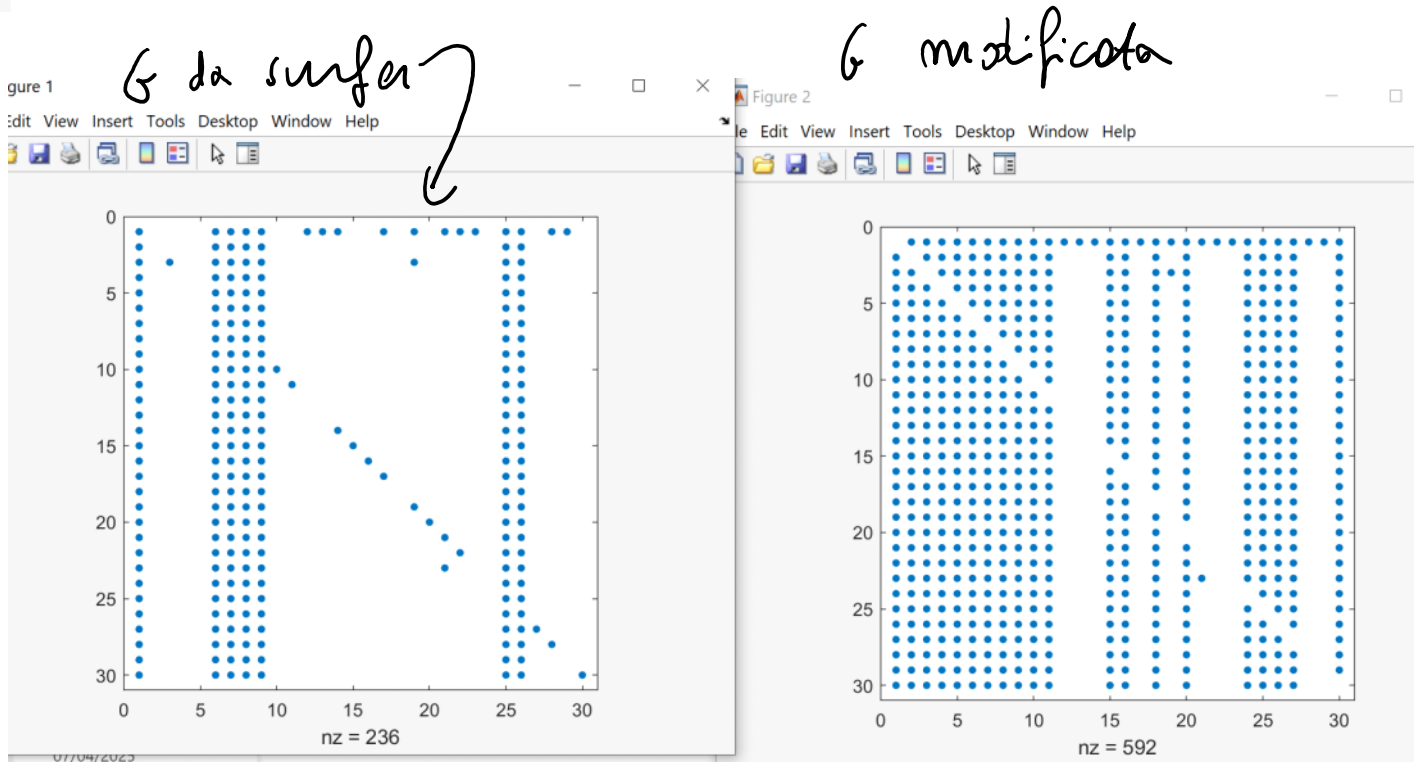
	1	2	3	4	5	6	7	8	9
1	0	1	0	0	0	0	0	1	0
2	1	0	1	1	0	0	0	0	0
3	1	1	0	0	0	0	0	0	0

	1	2	3	4	5	6	7	8	9
1	0	1	0	0	0	0	1	0	
2	1	0	1	1	0	0	0	0	
3	1	1	0	0	0	0	0	0	
4	0	1	0	0	0	0	0	0	
5	0	1	1	1	0	0	1	0	
6	0	1	0	1	1	0	0	1	
7	0	1	0	0	1	0	0	1	
8	0	1	0	0	1	1	1	0	
9									
10									

```

esercizioPageRank3.m  surfer.m  +
1 - [U,G] = surfer('https://www.harvard.edu',30);
2

```



prima

```
FILE NAVIGATE EDIT BREAKPOINTS RUN
esercizioPageRank3.m x pageRank.m x +
1 function [A,b]=pageRank(G,p)
2 n=size(G,1); delta=(1-p)/n;
3 I=eye(n);
4
5 G=G-diag(diag(G)); %eliminare eventuali link autoreferenzia
6 c=sum(G);
7 index=find(c==0); ni=length(index);
8 c(index)=(n-1)*ones(1,ni);
9 G(:,index)=ones(n,ni);
0 G=G-diag(diag(G)); %eliminare eventuali link autoreferenzia
1 figure(2)
2 spy(G)
3
4 D=diag(1./c);
5 e=ones(n,1);
6 A=I-p*G*D;
7 b=delta*e;
```

```
BREAKPOINTS RUN
esercizioPageRank3.m pageRank.m esercizioPageRank3.m x esercizioPageRank3.m x +
1 [Uweb,G] = surfer('https://www.harvard.edu',30);
2 p=0.85;
3 [A,b]=pageRank(G,p)
4 spy(G)
5
6 x=A\b
7 [y,index]=sort(x)
8
9
```

```

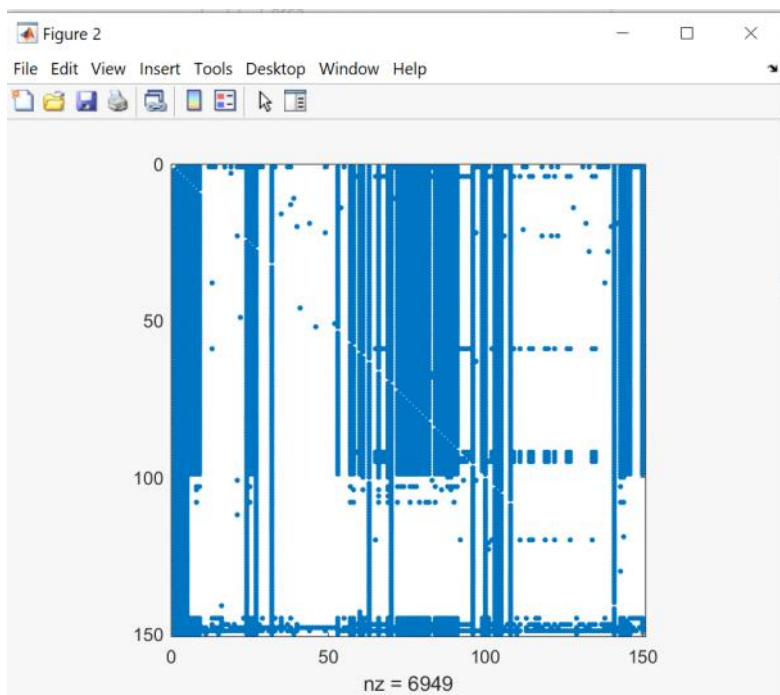
FILE NAVIGATE EDIT BREAKPOINTS RUN
esercizioPageRank4.m x +
1- |[Uweb,G] = surfer('https://www.harvard.edu',150);
2- p=0.85;n=size(G,1);
3- [A,b]=pageRank(G,p);
4-
5- tic
6- xMatlab=A\b;
7- [yMatlab,indexMatlab]=sort(xMatlab);
8- toc
9-
10- tic
11- [U,c,deter] = gaussNaive(A,b);
12- xN=backs(U,c);
13- [yN,indexN]=sort(xN);
14- toc
15-
16- tic
17- [U,c,deter,pivot] = gaussPiv(A,b);
18- xP=backs(U,c);
19- [yP,indexP]=sort(xP);
20- toc

```

```

21-
22- tic
23- x0=zeros(n,1);toll=1e-2;kmax=100;
24- [xJ,kJ,stimaerroreJ,ierJ]=jacobi(A,b,x0,toll,kmax);
25- [yJ,indexJ]=sort(xJ);
26- toc
27-
28- tic
29- [xG,kG,stimaerroreG,ierG]=gaussSeidel(A,b,x0,toll,kmax);
30- [yG,indexG]=sort(xG);
31- toc
32-

```



→ Non si sente dietro il comando
 a riga il metodo

Command Window

```
>> esercizioPageRank4
Elapsed time is 0.000956 seconds.
Elapsed time is 0.091965 seconds.
Elapsed time is 0.104500 seconds.
Elapsed time is 0.025480 seconds.
Elapsed time is 0.026260 seconds.
```

Non si mette dietro il comando
ci sia il metodo
di Gauss su
pivoting parziale
il più rapido
perché
ottimizzato
su matrici
sparse

→ NAIVE
→ PIV
→ JACOBI
→ GAUSS SEIDEL

pivoting è superfluo, più conveniente
NAIVE

NON FA SCAMBI

Iterativi più convenienti: nonostante nell'ultima
pagina abbiamo un ordinamento "scagliato" a causa
della $\text{toll} = 10^{-2}$, le pagine più importanti sono
ordinate bene (anche se il valore del ranking è
diverso)

yMatlab					indexMatlab				
150x1 double					150x1 double				
	1	2	3	4		1	2	3	4
34	19				133	11			
35	52				134	19			
36	14				135	52			
37	23				136	14			
38	51				137	23			
39	150				138	51			
40	93				139	150			
41	94				140	93			
42	95				141	94			
43	59				142	95			
44	92				143	59			
45	4				144	92			
46	147				145	4			
47	145				146	147			
48	1				147	145			
49	149				148	1			
50	148				149	149			
51					150	148			

	1	2	3
134	0.005932027656914		
135	0.006046365800076		
136	0.006190675352283		
137	0.006276115759430		
138	0.006335445161459		
139	0.008354534813580		
140	0.010668369613911		
141	0.012127520188912		
142	0.012456928699780		
143	0.014152221357265		
144	0.014809230923807		
145	0.017278444330762		
146	0.034657919514537		
147	0.084324810929322		
148	0.098403998621146		
149	0.105537557414269		
150	0.113470647420494		
151			

	1	2
134	0.005688833603027	
135	0.005774837705644	
136	0.005930700353777	
137	0.006023709884574	
138	0.006044522208487	
139	0.007985484280975	
140	0.010196055194777	
141	0.011575590697868	
142	0.011889420039843	
143	0.013493796784865	
144	0.014113271056685	
145	0.016451412542330	
146	0.033083893099567	
147	0.079563667947539	
148	0.093010160483425	
149	0.100188221739202	
150	0.107662430814106	
151		

osservazione: Gli iterativi non verificano $\sum x_i = 1$ ad ogni iterazione.

```
>> sum(xJ)
```

```
ans =
```

```
0.9544
```

```
>> sum(xMatlab)
```

```
ans =
```

```
1
```